

Qt/QML Advanced Integration Exercise

EXPOSING C++ MECHANICS, PROPERTIES, AND ENUMS TO QML ENGINE

Objective: Build a *Smart Home Automation Dashboard* backend and frontend structure to demonstrate proficiency in various methodologies of bridging C++ context, types, singletons, and properties over to the Qt Quick / QML engine architecture using modern Qt 6 APIs.

1. Registration Methods (<QQmlEngine>)

Your solution must properly register C++ classes using the standard registration functions or macros to expose types directly to the QML type system:

- `qmlRegisterType<T>()` : Register a C++ class named `SmartDevice` as an instantiable type in QML.
- `qmlRegisterSingletonInstance<T>()` : Expose a single, pre-existing C++ instance of a `SystemLogger` utility class, making it globally reachable as a singleton module in QML.
- `qmlRegisterUncreatableType<T>()` : Register a wrapper namespace/class called `DeviceEnums` containing a status enum, allowing QML to reference the enum values without permitting instantiation of the wrapper class itself.

2. Context Injections (Context Properties)

Aside from type registration, you must demonstrate the explicit injection of an object instance directly into the QML engine's context:

- Instantiate a `SystemConfig` configuration object within your C++ `main.cpp`.
- Inject this instance into the root context of your QML engine utilizing `setContextProperty()` under the context identifier name `systemConfig`.

3. Property Systems (Q_PROPERTY) & Enums

Inside your instantiable `SmartDevice` class, you must implement standard Qt properties that bind seamlessly into the QML engine declarative layer:

- **deviceName:** A read-only string property (`QString`).
- **deviceStatus:** A read-write property map linked to an underlying enum state. The enum must reside within your uncreatable `DeviceEnums` type, using proper Qt macros (`Q_ENUM` or `Q_ENUM_NS`).
- **batteryLevel:** A read-only integer property (`int`) that changes dynamically over time (e.g., simulated via a `QTimer`). Ensure a valid `NOTIFY` signal is supplied to force immediate declarative visual updates in QML when its value changes.
- **Methods:** Expose a mechanism to toggle the device power state via a `Q_INVOKABLE` function or slot named `togglePower()`.

4. Implementation Deliverables Checklist

C++ Architecture Setup

- Define **DeviceEnums** with states: **Off** , **On** , **Standby** , **Error** .
- Implement **SmartDevice** deriving from **QObject** with all matching property headers, getters, setters, and signals.
- Implement **SystemLogger** inheriting from **QObject** containing an invokable method: **void log(QString message)** .
- Implement **SystemConfig** containing a property string **homeName** .

Declarative Frontend (main.qml)

- Import the customized C++ registered URI namespace.
- Instantiate multiple instances of **SmartDevice** inline within the QML structure.
- Incorporate an interface showing names, live changing battery status lines, and the global context property **systemConfig.homeName** .
- Hook up a button to run the invokable **togglePower()** method on click.
- Invoke the registered singleton logger class inside a QML **onDeviceStatusChanged** event handler.
- Utilize the uncreatable enum type reference explicitly to drive conditional styling elements (e.g., matching a red border if state matches **DeviceEnums.Error**).